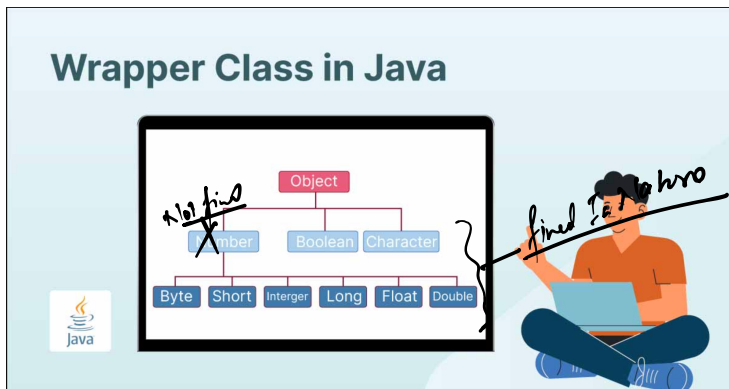


Cs Session 4

Monday, 23 February 2026 9:10 AM



long package is implicitly Imported in Java. String System

1 this Keyword in Java

◆ Definition

this is a reference variable that refers to the **current object of the class**. It is mainly used to differentiate **instance variables** from **local variables**.

◆ Why Do We Need this?

When local variables (constructor parameters) have the same name as instance variables, Java gets confused.

Example without this:

```
class Student {
    int id;

    Student(int id) {
        id = id; // Problem: refers to local variable
    }
}
```

Here, the instance variable is not initialized.

◆ Correct Version Using this

```
class Student {
    int id;

    Student(int id) {
        this.id = id; // Refers to current object variable
    }

    void display() {
        System.out.println("ID: " + id);
    }
}
```

```
public static void main(String[] args) {
```

```

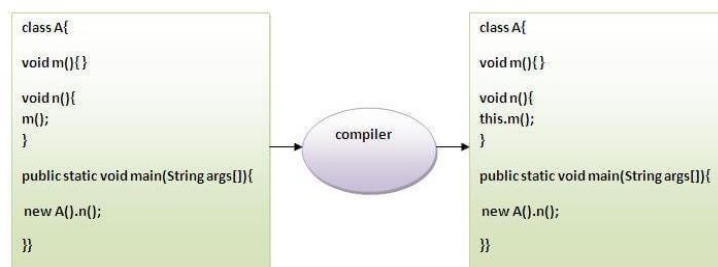
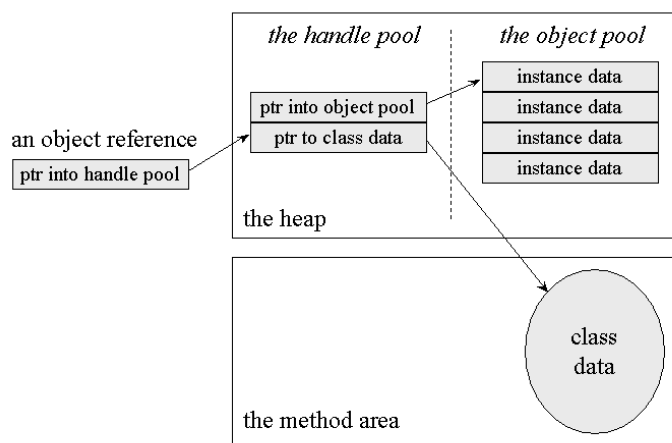
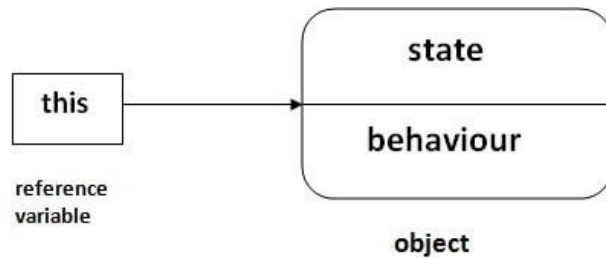
public static void main(String[] args) {
    Student s1 = new Student(101);
    s1.display();
}
}

```

 **Output:**

ID: 101

◆ Memory Representation of this



4

👉 this always points to the **current object in heap memory**.

◆ Uses of this

Use

Refer instance variable
Call current class method
Call current class constructor
Pass current object

Example

this.name = name;
this.display();
this(101);
method(this);

◆ Constructor Chaining Using this()

Output:

Parameterized Constructor

Default Constructor

👉 this() must be the **first statement** inside constructor.

2 super Keyword in Java

◆ Definition

super is used to refer to the **immediate parent class object**.
It is mainly used in **Inheritance**.

◆ Why Do We Need super?

When parent and child class have the same variable name, we use super to access parent variable.

◆ Example of super Variable

```
class Parent {
    int x = 100;
}

class Child extends Parent {
    int x = 200;

    void show() {
        System.out.println("Child x: " + x);
        System.out.println("Parent x: " + super.x);
    }

    public static void main(String[] args) {
        Child obj = new Child();
        obj.show();
    }
}
```

Output:

```
Child x: 200
Parent x: 100
```

◆ super Method Example

```
class Parent {
    void show() {
        System.out.println("Parent method");
    }
}

class Child extends Parent {
    void show() {
        super.show();
        System.out.println("Child method");
    }

    public static void main(String[] args) {
        Child obj = new Child();
        obj.show();
    }
}
```

Output:

```
Parent method
Child method
```

◆ super() Constructor Example

```
class Parent {
    Parent() {
        System.out.println("Parent Constructor");
    }
}
```

```

}

class Child extends Parent {
    Child() {
        super();
        System.out.println("Child Constructor");
    }

    public static void main(String[] args) {
        Child obj = new Child();
    }
}

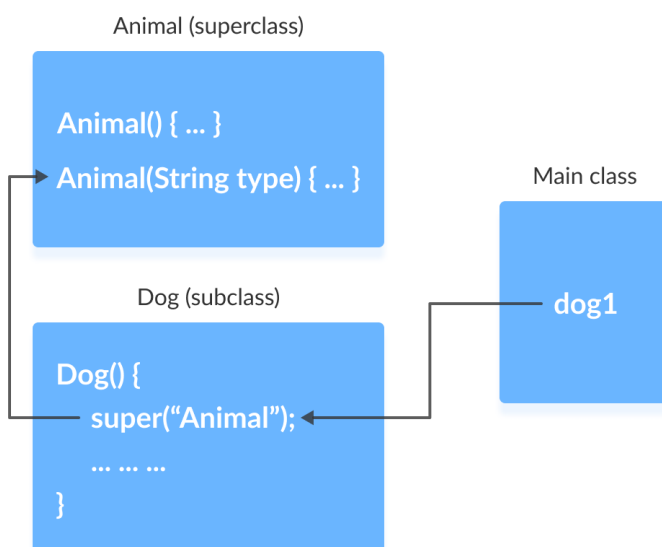
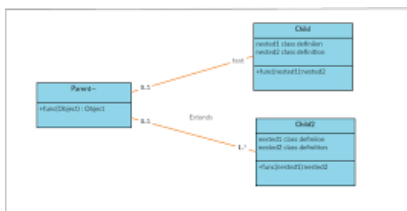
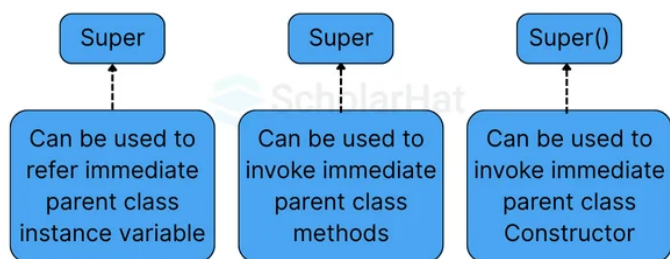
```

Output:

Parent Constructor
Child Constructor

◆ Working of super

Super Keyword in java



4

- ☞ `super()` is automatically called if not written explicitly.
- ☞ It must be the **first statement** in constructor.

3 Difference Between this and super

Feature	this	super
Refers to	Current class object	Parent class object
Used in	Same class	Child class
Access variable	Yes	Yes
Call constructor	this()	super()
Used in inheritance	No	Yes

4 Important Interview Points (For MCA / Placement)

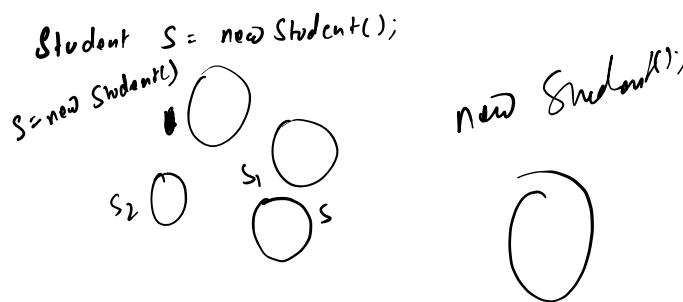
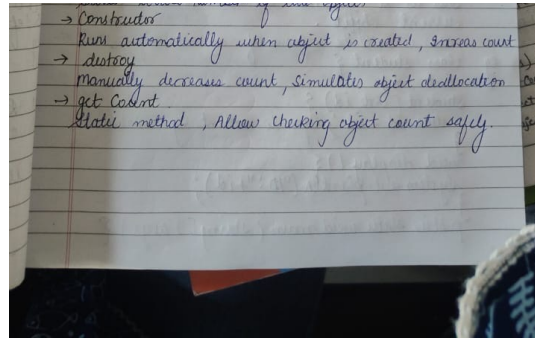
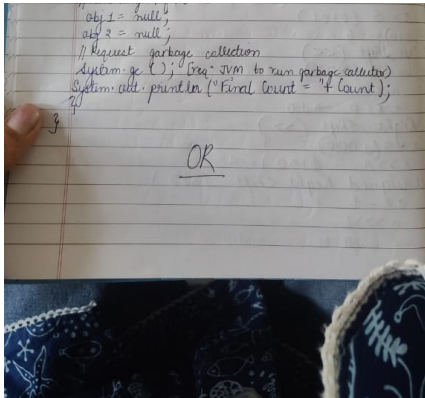
- ✓ this cannot be used in static context
- ✓ super cannot access private members of parent
- ✓ Constructor chaining improves code reusability
- ✓ If parent has no default constructor → child must use super(parameters)

Program

```
class ObjectCounter {
//static variable de count object
static int count = 0;
// constructor
ObjectCounter() {
    count++;
    System.out.println("Object Created-Count=" + count);
}
//finalize method (called before object is destroyed)
protected void finalize() {
    count--;
    System.out.println("Object Destroyed-Count=" + count);
}
public static void main(String[] args) {
    ObjectCounter obj 1 = new ObjectCounter();
    ObjectCounter obj 2 = new ObjectCounter();
    ObjectCounter obj 3 = new ObjectCounter();
//After some time...
```

```
class ObjectCounter {
    static int count = 0;
    ObjectCounter() {
        count++;
        System.out.println("Object Created-Count=" + count);
    }
    void destroy() {
        count--;
        System.out.println("Object Deallocated-Count=" + count);
    }
    public static void main(String[] args) {
        ObjectCounter obj 1 = new ObjectCounter();
        ObjectCounter obj 2 = new ObjectCounter();
        obj 1. destroy();
        obj 2. destroy();
        System.out.println("Final Count = " + count);
    }
}
→ static int count
• shared among all objects
(there is only one count for all objects)
```

Count 0



1 What is Exception?

◆ Definition

An **Exception** is an unwanted or unexpected event that occurs during program execution and disrupts the normal flow of the program.

👉 In Java, exceptions are objects.

◆ Example Without Exception Handling

```

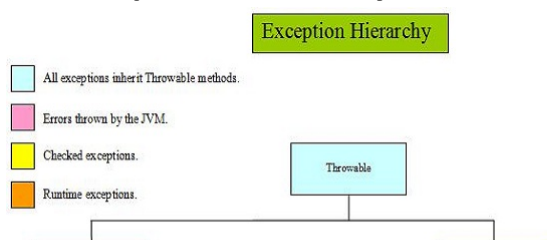
class Demo {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        int c = a / b; // ArithmeticException
        System.out.println(c);
    }
}

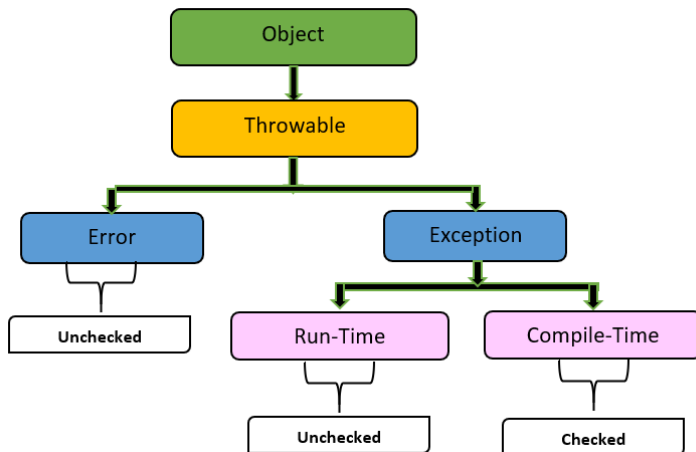
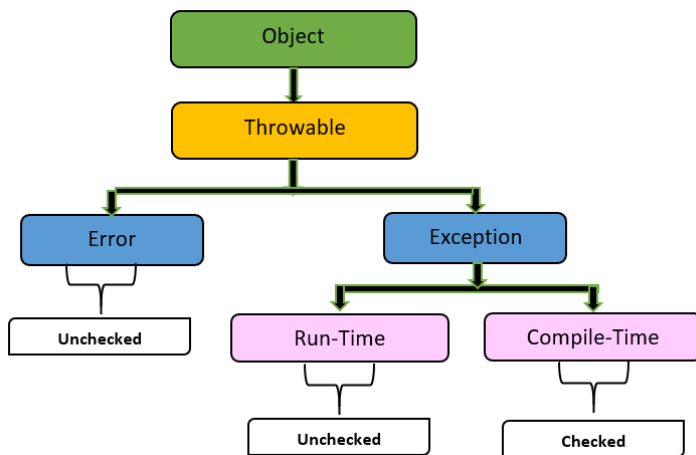
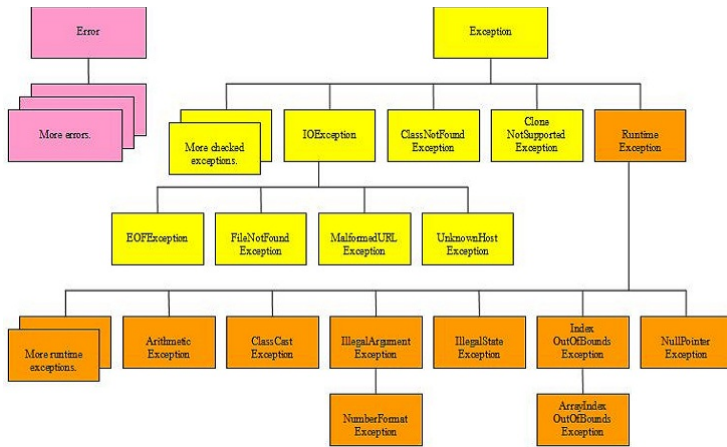
```

✗ Output:

Exception in thread "main" java.lang.ArithmeticException: / by zero
 Program terminates abnormally.

2 Exception Hierarchy in Java





4

◆ Main Classes

- **Throwable** (Parent Class)
 - Error
 - Exception

◆ Types of Exceptions

1 Checked Exception

- Checked at compile time
- Must be handled
- Example: IOException, SQLException

2 Unchecked Exception

- Checked at runtime
- Example: ArithmeticException, NullPointerException

3 try-catch Block

◆ Syntax

```
try {  
    // risky code  
} catch (ExceptionType e) {  
    // handling code  
}
```

◆ Example

```
class Demo {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            int c = a / b;  
            System.out.println(c);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

✓ Output:

Cannot divide by zero
Program continues...

👉 Program does NOT terminate abnormally.

4 Multiple catch Block

```
class Demo {  
    public static void main(String[] args) {  
        try {  
            int arr[] = new int[3];  
            arr[5] = 10;  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Arithmetic Error");  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index Error");  
        }  
    }  
}
```

5 finally Block

◆ Definition

finally block always executes whether exception occurs or not.

Used for:

- Closing files
- Database connections
- Resource cleanup

◆ Example

```
class Demo {  
    public static void main(String[] args) {  
        try {  
            int x = 5;  
            System.out.println(x);  
        }  
        catch (Exception e) {  
            // ...  
        }  
        finally {  
            // ...  
        }  
    }  
}
```



```

        System.out.println("Exception occurred");
    }
    finally {
        System.out.println("Finally block executed");
    }
}
}

```

6 throw Keyword

Used to manually throw an exception.

```

class Demo {
    public static void main(String[] args) {
        int age = 15;

        if(age < 18) {
            throw new ArithmeticException("Not eligible to vote");
        }
    }
}

```

7 throws Keyword

Used to declare exception in method signature.

```

import java.io.*;

class Demo {
    static void readFile() throws IOException {
        FileReader file = new FileReader("test.txt");
    }

    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) {
            System.out.println("File not found");
        }
    }
}

```

8 Custom Exception

```

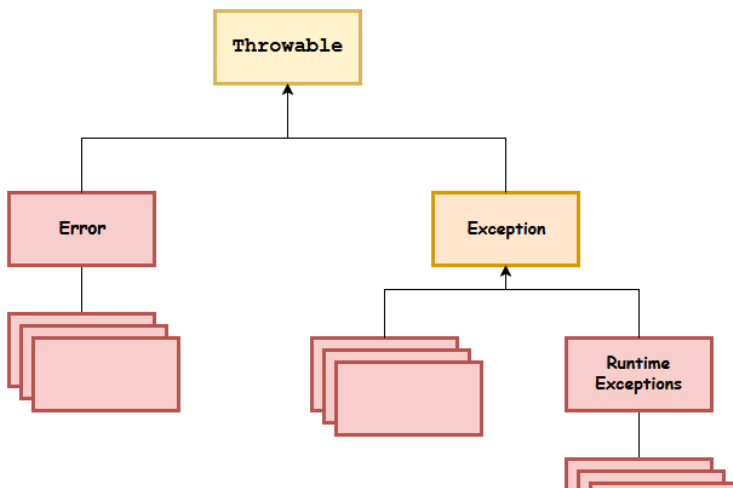
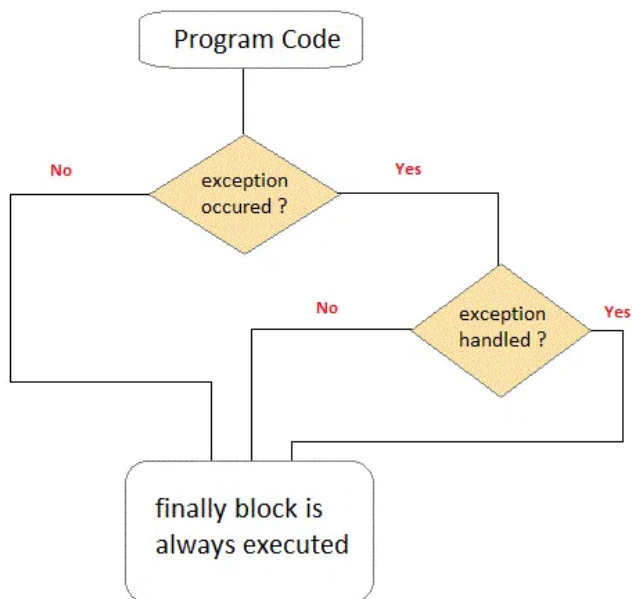
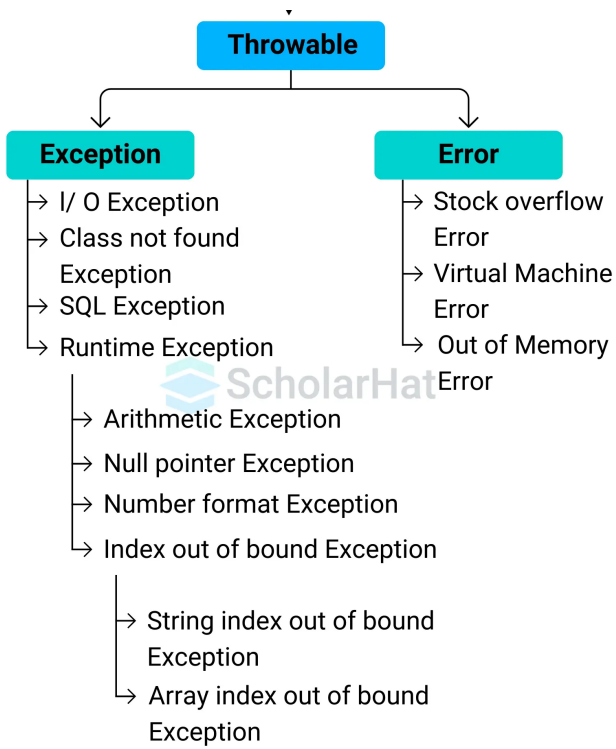
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}

class Demo {
    public static void main(String[] args) {
        try {
            int age = 15;
            if(age < 18) {
                throw new InvalidAgeException("Age must be 18+");
            }
        }
        catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

9 Flow of Exception Handling

Object
↓





10 Important Interview Points (For MCA / Placement)

- ✓ Checked exceptions must be handled or declared
- ✓ finally block always executes (except System.exit())
- ✓ Only one exception executes at a time
- ✓ Catch blocks must be ordered from specific → general
- ✓ Custom exception must extend Exception class

Difference: throw vs throws

Feature	throw	throws
Used for	Throw exception	Declare exception
Used inside	Method body	Method signature
Keyword count	One exception	Multiple allowed